

Procesamiento de Datos Masivos

Clase 11 - RAG

Qué haremos hoy

- Aplicación de bases de datos de vectores
- Cómo y cuándo usar RAG
- Aprender a usar Qdrant

Contextualización: LLMs y datos hoy





LLMs y datos hoy

- LLMs (redes neuronales y tokens)
- Etapa de entrenamiento: usamos los datos para que la red ajuste pesos y pueda predecir de mejor manera próximos tokens a escribir
- Especialmente útiles para responder “cosas que ya sepa”

LLMs y datos hoy

Pero...

- Problema 1: alucinaciones
- Problema 2: ventanas de contexto
- Problema 3: datos privados y nuevo conocimiento post entrenamiento

LLMs y datos hoy

Podemos:

	Re-entrenar	Fine-tuning	Contexto (prompt)
Costo	Muy alto Millones USD por ciclo	Medio-alto Miles a decenas de miles	Bajo Solo tokens de API
Actualización	Lenta Semanas o meses	Días o semanas	Instantánea
Datos privados	Riesgoso Pipeline complejo, difícil de aislar	Posible Rígido y costoso de mantener	Directo Se pasan en el prompt
Escalabilidad	No escala No sirve con info diaria	Limitada Requiere re-entrenamiento frecuente	Limitada Acotada por ventana de contexto
Riesgo principal	Degrada conocimiento previo	Puede "olvidar" capacidades	Se "marea" con mucho contexto

Entonces

- Los LLMs son poderosos, pero solo saben lo que aprendieron en el entrenamiento
- Reentrenar y fine-tuning son lentos y caros para información que cambia constantemente
- Meter todo en el contexto tiene límites duros de espacio y precisión

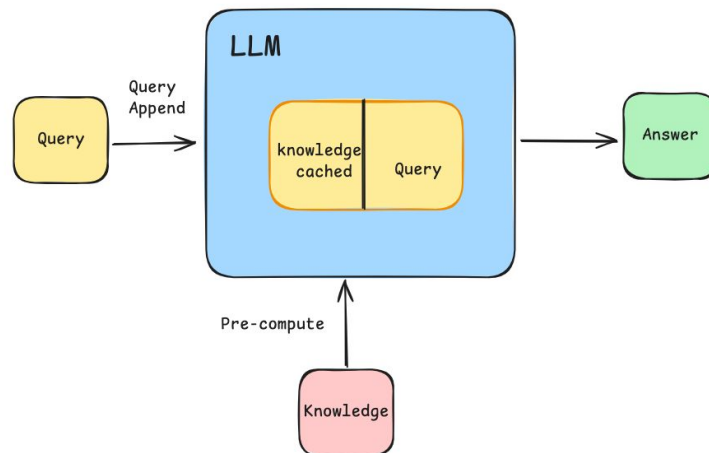
Cache-Augmented Generation (CAG)

CAG

- Sabemos que si le entregamos más contexto a los LLMs nos pueden responder mejor
- Estrategia basada en tener un corpus preprocesado sobre el que podremos obtener mejores respuestas a nuestro dominio
- Este es un “resumen” del contenido de nuestros documentos que cabe en nuestra ventana de contexto

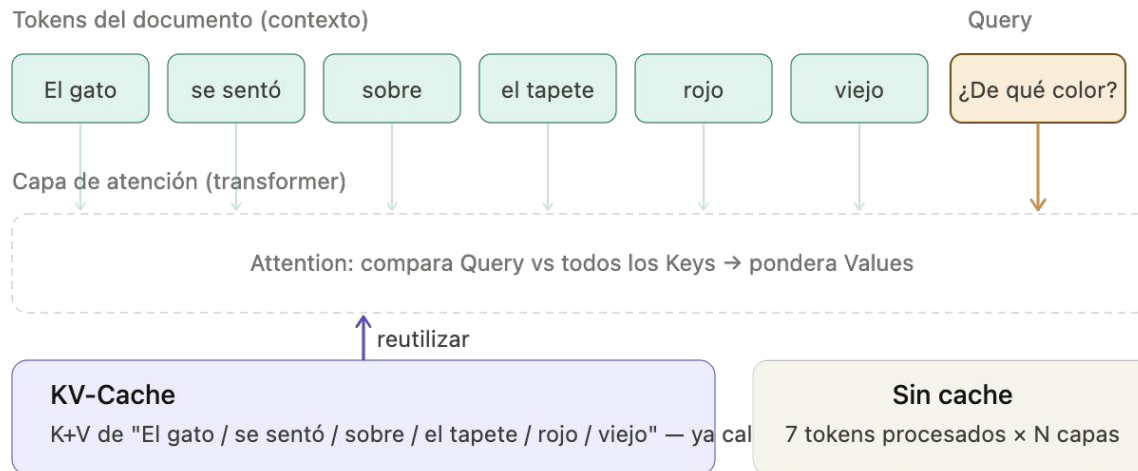
CAG

- Beneficios: liviano, rápido, barato y simple de implementar
- Podemos entregar así el contexto de nuestro interés y mejorar respuestas
- Algunos casos de uso:
 - Manuales de usuario
 - Sistemas de atención al cliente
 - Datos que no cambian tanto



CAG

- ● ● ● Sin cache: procesando todo desde cero



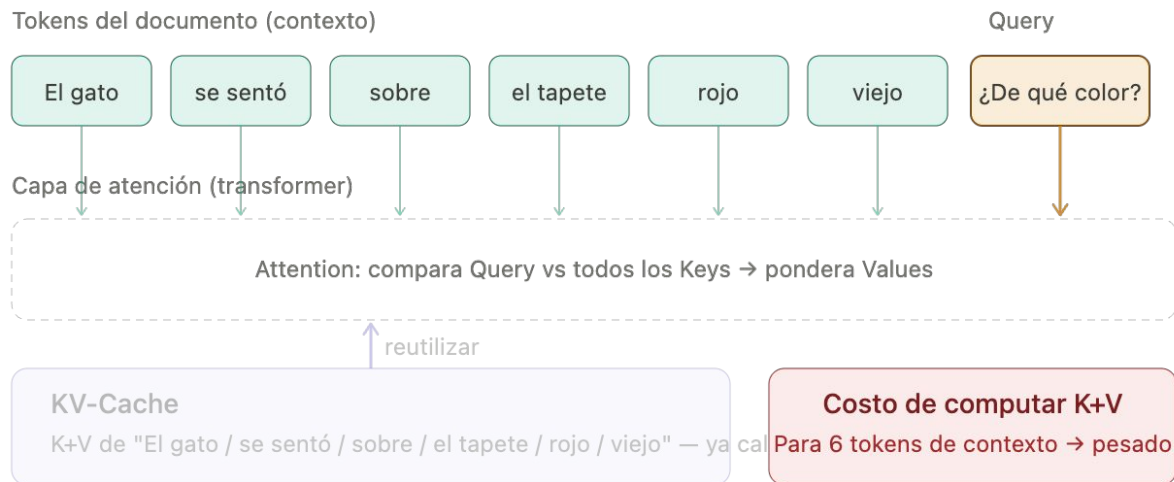
← Anterior

Siguiente
→

Cada vez que llega una query, el modelo procesa todos los tokens del contexto

CAG

- ● ● ● El modelo calcula K y V para cada token de contexto



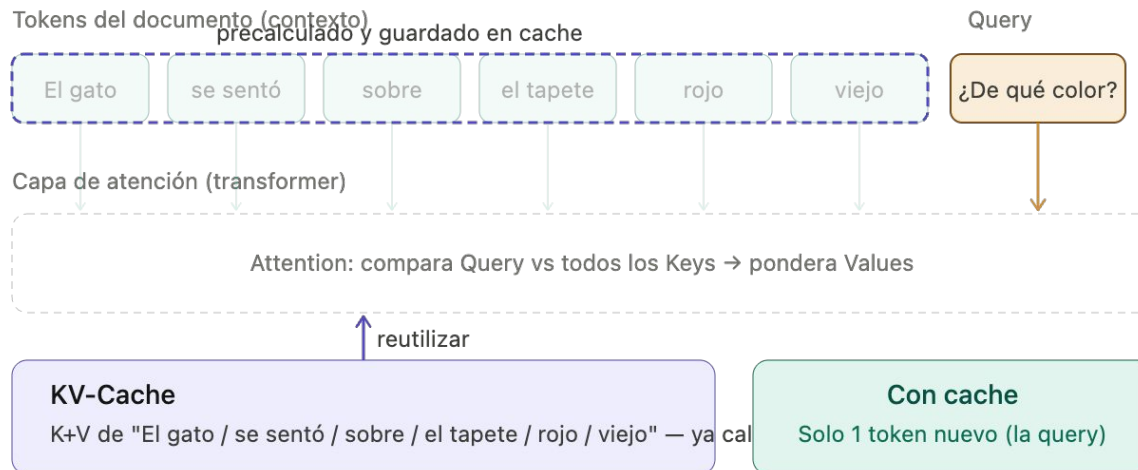
← Anterior

Siguiente →

K (Key) y V (Value) son representaciones que capturan el significado de cada token

CAG

- ● ● ● Con KV-cache: guardamos esos cálculos



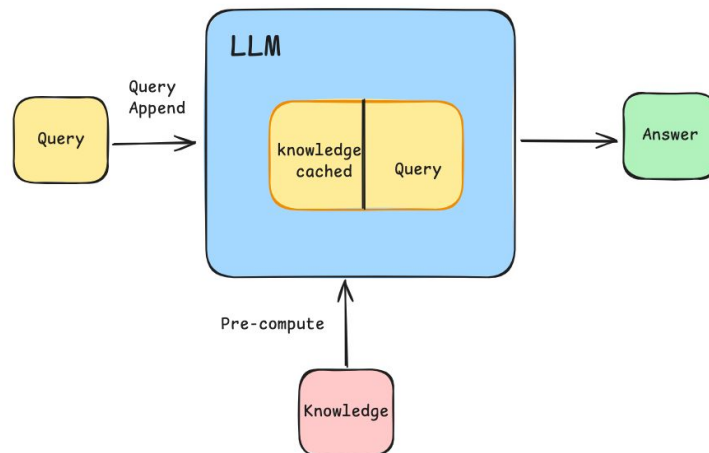
← Anterior

Siguiente →

K y V del contexto no cambian entre queries → los guardamos en memoria

CAG - tradeoff

- Estamos limitados por la ventana de contexto
- Si la información que tenemos es mucha, no podremos aprovecharla adecuadamente
- Necesitaremos reprocesar el cache si queremos agregar nueva información



CAG - tradeoff

- Si queremos obtener la fuente original de la respuesta
- Si queremos usar datos que se agregan con mucha frecuencia
- Si queremos búsquedas de mayor complejidad

Para todo esto, CAG nos queda chico

Retrieval-Augmented Generation (RAG)





RAG

- Nos basamos en la idea de buscar información relevante que ayude al LLM a responder mejor
- Nuestra fuente de datos es independiente del LLM
- Etapa de **procesamiento/ingesta** y de **consumo**

RAG

Ingredientes:

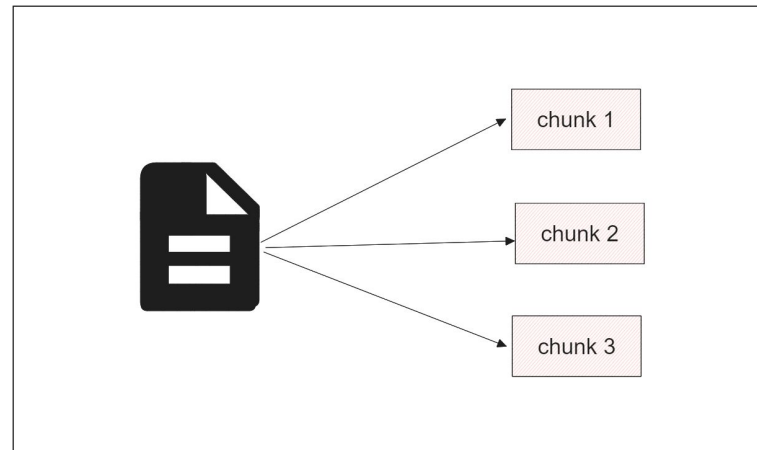
- Datos
- Splitter o chunking
- Embedder
- DB vectorial
- LLM
- Retriever

RAG - Datos

- Imágenes, PDF, texto plano...
- La calidad del RAG depende directamente de la calidad de los datos

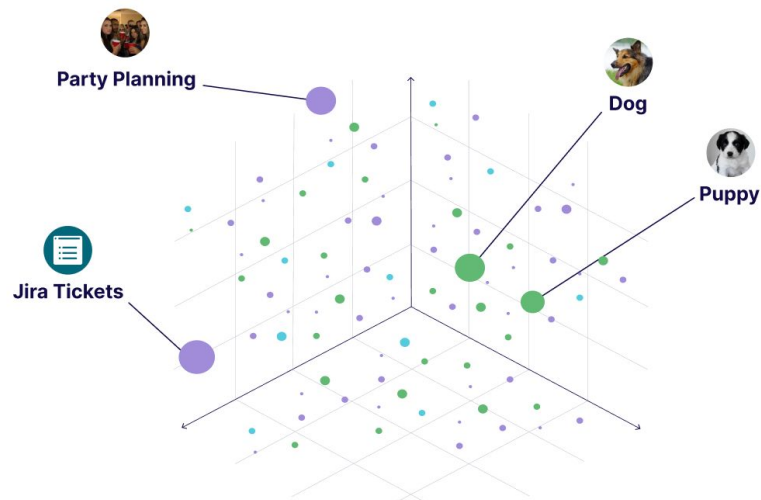
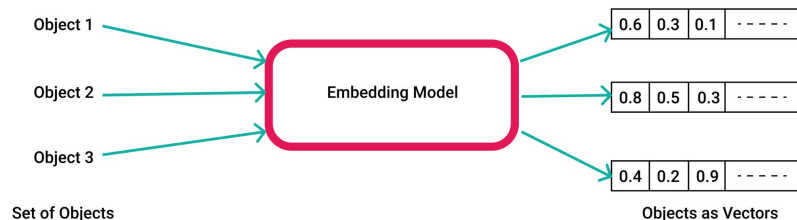
RAG - Splitter

- Los embedders tienen una ventana de contexto limitada
- Solución: dividir en fragmentos (chunks)
- Overlap
- Diferentes estrategias



RAG - Embedder

- Transformación de datos a vectores semánticos
- Textos similares quedan cerca en el espacio vectorial; distintos quedan lejos
- Vectores densos, sparse y representación híbrida
- ◆ Dimensiones

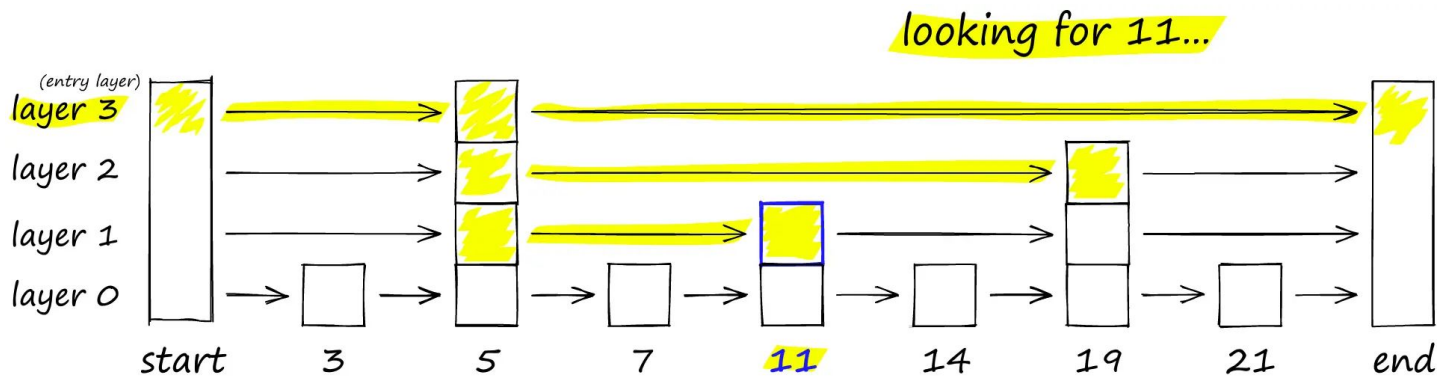


RAG - DB Vectorial

- Almacena los vectores y permite buscar los más cercanos eficientemente
- Distancia para vectores sparse y dense
- Algunas opciones: Qdrant, PGVector (sobre Postgres), Chroma
- Índices ANN (approximate nearest neighbors), como **HNSW**

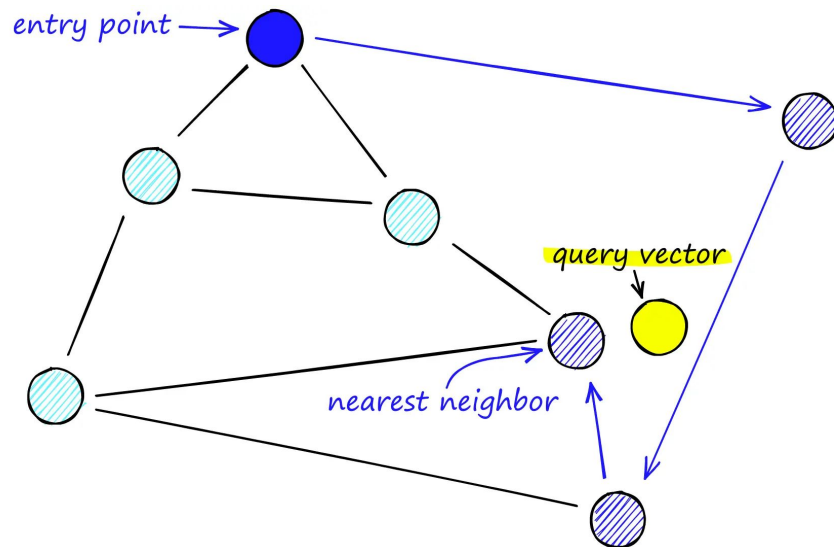
Hierarchical Navigable Small World

- Idea: **Skip List** + Navigable Small World Graphs



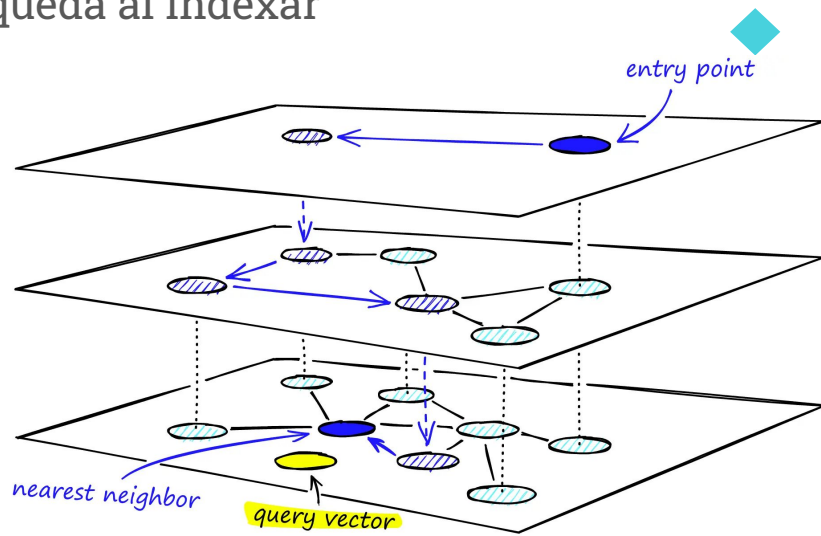
Hierarchical Navigable Small World

- Idea: Skip List + **Navigable Small World Graphs**



Hierarchical Navigable Small World

- Idea: Skip List + Navigable Small World Graphs
- Parámetros:
 - M : Conexiones por nodo por capa
 - $ef_construction$: Amplitud de búsqueda al indexar
 - ef_search : Amplitud de búsqueda al consultar
- De esta forma, encontramos vectores aproximados de forma eficiente en tiempo, a costa de RAM e inserciones más lentas



RAG - Retriever

- Dado un query, busca los chunks más relevantes en la BDD vectorial
- Decisiones: cuantos chunks recupero, cómo ranqueo resultados, estrategia de búsqueda
- *Reciprocal Rank Fusion* (RRF) en búsqueda híbrida

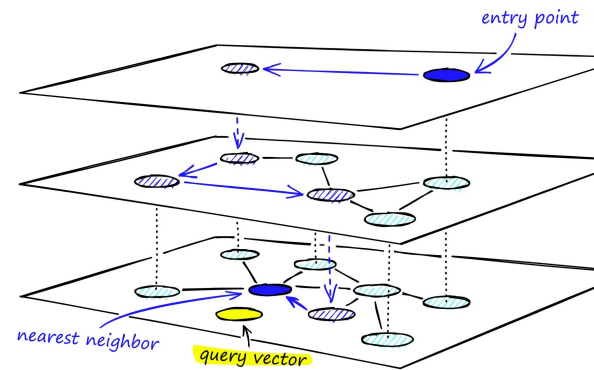
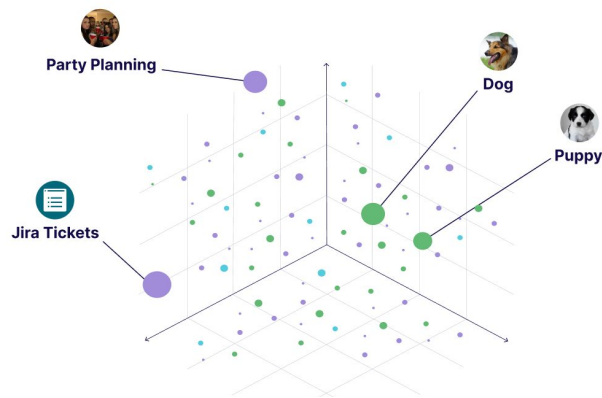
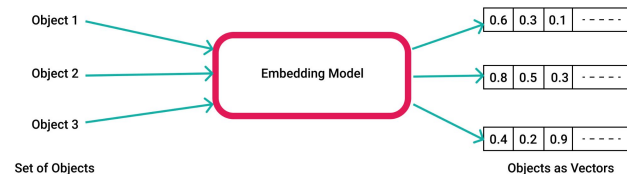
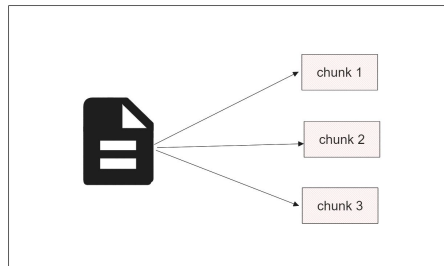
RAG - LLM

- El componente que genera la respuesta final usando los chunks recuperados como contexto
- No "sabe" la respuesta, la construye a partir de lo que el retriever le entrega
- La calidad de la respuesta depende tanto del LLM como de la calidad del contexto recuperado

RAG

Ingredientes:

- Splitter o chunking
- Embedder
- BDD vectorial
- LLM
- Retriever
- Datos



Arquitectura RAG

Cómo conectamos todo para que funcione

Ingesta

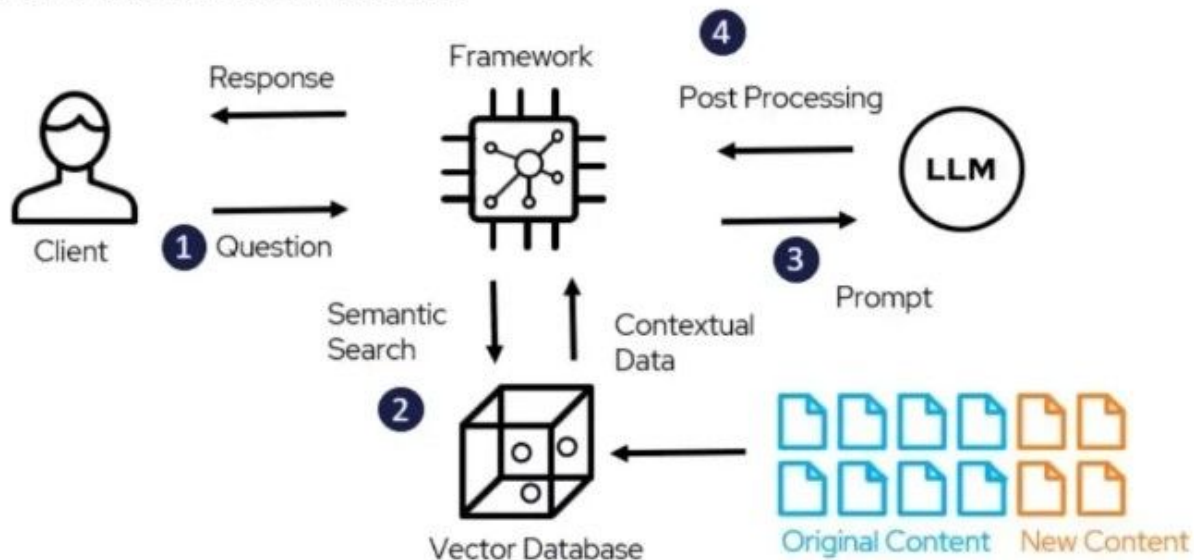
- Datos
- Splitter o chunking
- Embedder
- DB vectorial
- Adicional:
 - OCR
 - Tokenizer
 - Modelo atributos/properties

Consumo

- Embedder
- DB vectorial (con datos cargados)
- LLM
- Retriever
- Adicional:
 - Prompts
 - Rol del LLM
 - Guardrails

Arquitectura (simplificada)

RAG Architecture Model



Cerrando ideas



¿Cuándo usamos RAG o CAG?



- Simplicidad vs potencial
- Cantidad de información
- Frecuencia de actualización de datos
- Contexto



Yendo un paso más allá

- Function tools y agentes
 - Filtros automáticos
 - Conocimiento de dominios
 - Agentes especializados / multiagentes
- Consulta a datos estructurados y otras formas de información
- Embeddings multimodales
- Cómo entregamos la información
 - Calidad de respuestas
 - Historial
 - Uso de tokens

Ejercicio 1

Tengo una aplicación compleja con muchos módulos y funcionalidades.

¿Qué usarían para construir un chatbot que ayude a responder preguntas sobre cómo usar apropiadamente esta aplicación?

Ejercicio 2

Contamos un conjunto de PDFs a los que queremos hacer RAG.
Construir un sistema que permita a los usuarios:

- Hacer **consultas de alto nivel** (semánticas) respondidas con el contenido de los PDF
- **Obtener las fuentes** que se usaron para respuesta
- Poder **cargar nuevos documentos** para ser consumidos por la comunidad

Ejercicio 2.2

¿Qué cambios son necesarios para poder hacer búsquedas por palabras clave?

Hay que jugar con los datos y configuraciones



- ¿Cuántos chunks recupero?
- Se debe de iterar con los prompts para lograr mejores resultados
- Un buen modelo de datos te puede ayudar a escalar y lograr responder preguntas más complejas
- ¿Cómo pueden saber si los cambios mejoraron o empeoraron resultados?

Lectura complementaria

- HNSW: <https://www.pinecone.io/learn/series/faiss/hnsw/>
- Multimodal embedding - gemini-embedding-2: <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/embedding-2>
- <https://qdrant.tech/documentation/>